Lossless Compression of Proximity Graph Indexes for Efficient Approximate Nearest Neighbors Search

Francesco Benocci[1,*,†], Leonard Dahlmann[2,†], Francesco Gargiulo[3,†], Tobias Heuer[4,†], Silvio Martinico[1,3,†], Franco Maria Nardini[1,†], Cosimo Rulli[1,†] and Rossano Venturini[3,†]

[1]*ISTI-CNR, Pisa, Italy*
[2]*eBay Inc., Aachen, Germany*
[3]*University of Pisa, Pisa, Italy*
[4]*eBay Inc., Karlsruhe, Germany*

Abstract

Graph-based Approximate Nearest Neighbor (ANN) indexes, such as HNSW, are the backbone of modern vector search. However, their massive memory footprint creates a severe infrastructure bottleneck. While vector quantization is widely adopted to reduce the memory required to store data vectors, the graph's edge structure increasingly dominates total memory usage. Compressing these adjacency lists for online search presents a critical system challenge: because decompression occurs during query processing, i.e., while navigating the graph, memory savings cannot come at the expense of query latency. In this paper, we present a compression technique explicitly tailored for live HNSW routing. By applying offline graph permutations to artificially induce structural locality, we enable the use of SIMD-accelerated integer compressors, such as Stream VByte, for rapid on-the-fly decompression. Evaluated across diverse real-world workloads, we significantly outperform the current state of the art in both compression ratio and query latency. Our experimental results show that we achieve compression ratios up to 52.7% with negligible penalties in latency compared to the uncompressed baseline. Furthermore, our technique has been effectively deployed in production at eBay, achieving up to 44.3% reduction in edge storage with no degradation in query latency. This extended abstract summarizes results published at CIKM 2026 [1].

Keywords

Approximate Nearest Neighbors Search, Graphs, Compression

1. Introduction

Modern retrieval systems rely on Approximate Nearest Neighbor (ANN) methods, and on Hierarchical Navigable Small World (HNSW) graphs [2, 3, 4] in particular, to search high-dimensional vector embeddings efficiently. These indexes are typically designed to reside entirely in main memory. Vector quantization techniques [5, 6, 7] successfully reduce the memory footprint of the stored vectors, so the graph structure (the adjacency lists) takes an increasing share of the total memory consumption [8]. Compressing adjacency lists for online search is a system problem rather than an archival one: decompression occurs on-the-fly during query routing, so any latency overhead is paid on every query. In this work, we provide a thorough exploration of the structural properties of HNSW graphs and introduce a compression framework tailored for live routing. Our approach relies on two key components. First, we employ offline node permutation techniques [9, 10] to artificially induce structural locality, which drastically improves the compressibility of the adjacency lists (Section 3). Second, we apply Stream VByte [11] compression, whose byte-aligned layout enables branch-free, SIMD-accelerated decoding at negligible cost at query time (Section 4). We evaluate our framework across four real-world

IIR2026: Italian Information Retrieval Workshop, November 12-14, 2026, Rome, Italy
*Corresponding author.
†All authors contributed equally to this research.
francesco.benocci@isti.cnr.it (F. Benocci); fdahlmann@ebay.com (L. Dahlmann); francesco.gargiulo@phd.unipi.it (F. Gargiulo); theuer@ebay.com (T. Heuer); silvio.martinico@phd.unipi.it (S. Martinico); francomaria.nardini@isti.cnr.it (F. M. Nardini); cosimo.rulli@isti.cnr.it (C. Rulli); rossano.venturini@unipi.it (R. Venturini)
0009-0001-5780-8954 (F. Benocci); 0009-0008-5881-8776 (L. Dahlmann); 0009-0000-5884-4298 (F. Gargiulo); 0000-0002-5399-0496 (T. Heuer); 0009-0005-7280-6147 (S. Martinico); 0000-0003-3183-334X (F. M. Nardini); 0000-0003-0194-361X (C. Rulli); 0000-0002-9830-3936 (R. Venturini)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

workloads ranging from 1M to ≈9M vectors, which comprise classical and neural datasets (SIFT1M, CoCondenser, SnowFlake, eBayProducts7M), dense and sparse vectors, and compressed and uncompressed representations. Our approach reduces edge space by up to 52% (SIFT1M) while query latency stays within a few percent of the uncompressed baseline. The system is currently deployed in production at eBay, in its search engine Cassini [12]: since its deployment in early 2025, we have measured a 44.3% reduction in the HNSW graph's memory footprint at identical ANN recall, with no measurable latency degradation under live production workloads.

2. Background

Given a dataset of N vectors in a D-dimensional vector space, the k-Approximate Nearest Neighbors (kANN) problem consists of efficiently retrieving the k closest points to a query vector under a given similarity measure, trading exact retrieval accuracy for improvements in query latency [13, 14]. HNSW [2] is one of the most effective and widely-adopted ANN indexes. It organizes the dataset into a multi-level proximity graph, where higher and sparser levels locate the query neighborhood and the lowest level, the *ground level*, contains all the points and refines the results. M bounds the number of connections per node, *efConstruction* sets the candidate pool at build time, and *efSearch* the exploration breadth at query time; our implementation adds a distance-adaptive early-termination threshold λ [15].

Adjacency lists can be arbitrarily ordered, so sorting them turns each list into a monotone integer sequence that an encoder can exploit, either directly, as in Elias–Fano [16], or after gap encoding [17, 18]. Entropy coders [19, 20] and universal codes [21, 22] are compact but decode bit by bit, a cost that online routing cannot afford, which is why latency-sensitive pipelines adopt byte-aligned compressors. Stream VByte [11] splits the encoded sequence into a *control stream*, where two bits per value give the byte length of each integer, and a *data stream* with the concatenated representations, so the decoder reads control bytes independently and accesses the data stream branch-free. A third option is list-level compression, where part of a list is encoded as a reference to another, as in WebGraph [23]. Severo *et al.* [8] recently studied lossless identifier compression for IVF [24] and graph ANN indexes, and found bit-packing and Elias–Fano to be the best options under online random access.

3. Analysis of HNSW Graphs

Consistent with our focus on practical deployments, we restrict our analysis to low-degree HNSW configurations, which in our benchmarks already achieve high recall. Prior work evaluates degrees from 16 to 256 [8]; low-degree graphs are a more challenging and practically relevant target, because shorter adjacency lists leave less redundancy to exploit and their per-list decompression cost weighs more against the work done per query. In this regime the edges are far from a marginal cost: they take between 33.8% and 57.1% of the index, and the graph dominates the vector data in quantized (SnowFlake) and low-dimensional (eBayProducts7M) settings.

Two structural properties drive our design choices. The first is the overlap between adjacency lists, which decides whether reference-based compression is profitable. HNSW enforces a strict degree bound on every node, unlike web and social networks, whose power-law degree distributions produce the high-degree hubs that reference-based compressors exploit: out-degrees in CoCondenser and eBayProducts7M concentrate near the cap of 64 edges per node. We derive an upper bound on the savings of reference copying that allows unbounded reference chains and ignores the cost of encoding the reference ID, so no implementation can reach it. On eBayProducts7M a profitable reference exists for over 99.9% of the nodes, yet the aggregate ceiling is 8.8%: an average list holds 49.9 edges (1147 bits) and shares only 5.2 of them with its optimal reference, worth ≈120 bits against a selection cost of ≈19 bits. Since a real implementation would also pay for chasing reference chains under random access, we exclude this family from further evaluation.

The second property is the distribution of the values inside the lists, which we characterize through the zero-order empirical entropy H_0 computed over all nodes. HNSW lists are close to uniform over the

Table 1
Average query time (μs) at different Accuracy@10 cut-offs (90, 94, 98). The space for each compression method is reported as bits/edge (b/e), with the compression ratio over Plain in gray.

	CoCondenser				eBayProducts7M				Sift1M				SnowFlake (PQ)			
	b/e	90	94	98	b/e	90	94	98	b/e	90	94	98	b/e	90	94	98
Plain	33.4	311	386	778	33.3	185	264	632	34.1	**55**	**69**	**111**	33.6	261	302	497
Bitpacker	24.1 1.3×	315	391	791	23.1 1.4×	193	277	668	20.2 1.6×	57	71	115	24.1 1.3×	266	310	509
Elias-Fano	23.6 1.4×	332	415	843	23.2 1.4×	231	333	832	20.7 1.6×	79	89	137	23.9 1.4×	288	338	563
Zeta	18.7 1.7×	327	406	819	17.4 1.9×	214	302	714	14.6 2.3×	70	87	141	17.0 1.9×	277	324	545
SVB	19.4 1.7×	**309**	**384**	**765**	18.3 1.8×	**177**	**250**	**564**	16.1 2.1×	58	71	114	18.0 1.8×	**260**	**302**	**494**

ID space, and bit-packing exceeds H_0 by at most 1.23 bits, so fixed-width packing is nearly optimal and little is left for an entropy coder. Gap encoding lowers H_0 from 22.77 to 18.69 bits on CoCondenser and from 22.62 to 18.52 bits on eBayProducts7M, a meaningful but insufficient reduction, since the gaps still span a large fraction of the ID space. The rest must come from the ID assignment itself: nodes that co-occur in adjacency lists should receive numerically close IDs. Both reordering algorithms we consider concentrate the edge mass along the diagonal of the adjacency matrix and cut the gap entropy substantially, to 16.02 bits (CoCondenser) and 14.99 bits (eBayProducts7M) with EGB [9], and to 15.86 and 14.90 bits with KaMinPar [10]. We adopt EGB, whose gain-based refinement is tailored to minimize intra-list gaps rather than a general k-way edge-cut objective, and whose native Rust implementation fits our indexing pipeline. Reordering also improves the spatial locality of the vectors, and it reduces the average query time of the uncompressed baseline from 3.0% (CoCondenser) to 17.0% (eBayProducts7M), in line with the cache and prefetching gains reported by Coleman *et al.* [25]. Its cost is the inverse permutation, which we store to report results under the original dataset IDs.

4. Compression Framework

The analysis above fixes our pipeline. We permute the node IDs offline with EGB, gap-encode the adjacency lists and compress them with Stream VByte, which gives up less than 1 bit/edge with respect to the most compact universal code we evaluated, Zeta [22], in exchange for branch-free SIMD decoding. The offsets that delimit the neighborhoods in kANNolo's flat neighbor array form a monotone sequence, and we compress them with Elias–Fano [16], which brings them from 64 to roughly 8 bits per element while it still serves the random accesses that query processing performs.

We implement the framework, and all evaluated baselines, within the HNSW index of the kANNolo [26] library, a highly optimized uncompressed baseline that ensures that any measured overhead is genuine rather than masked by inefficiencies of the underlying implementation. An abstraction layer isolates the physical representation of the adjacency lists from the logical traversal, so the search algorithm is unmodified and compressed and uncompressed backends are interchangeable. Compression is applied at the granularity of individual lists rather than over a flattened global array: decoding entry points are aligned with list boundaries, so each neighborhood is decompressed on demand and no unrelated neighborhood is scanned. Decompressed neighbors are buffered into small stack-allocated chunks, which removes temporary heap allocations and repeated decoding. Traversal operates natively in the reordered identifier space, since vectors are permuted along with the node IDs offline, and only the results are translated back to the original IDs.

5. Experiments

Datasets We experiment on four datasets that span image descriptors, text passages encoded with dense and sparse models, and product embeddings. Sift1M [27] is the most adopted dataset for ANN benchmarking [3], with 1M 128-dimensional descriptors evaluated with Euclidean distance on 10K queries. From Ms Marco-v1 [28], a collection of ≈8.8M text passages with 6,980 queries (`dev.small`),

we produce two embedding sets and evaluate both under inner product similarity. SnowFlake [29] is a lightweight dense encoder producing 1024-dimensional vectors; since its encoding of MsMarco exceeds 30 GB, we compress it via Product Quantization [5] with $M = 128$ subspaces and adopt a two-stage search, where the top-k' ($k' > k$) candidates retrieved on PQ-encoded vectors are reranked with the uncompressed representations. CoCondenser [30] is a sparse encoder built on the BERT [31] vocabulary, whose vectors live in an ≈30K-dimensional space and have 120 non-zero components on average. eBayProducts7M is a private set of roughly 7M 96-dimensional product embeddings from eBay, produced by an in-house BERT variant fine-tuned for e-commerce product search, with 10K queries. The four ground-level graphs hold between 30M and 400M edges, with 30.0 to 49.9 edges per node.

Setup We measure the memory footprint as the average number of bits per edge, which accounts for the neighbor lists on all HNSW levels, the offsets array, and, where enabled, the reverse mapping of the permutation. Effectiveness is Accuracy@10, the fraction of the true 10 nearest neighbors among the top-10 retrieved results, averaged over the queries. Efficiency is the average query latency of the fastest configuration that reaches a given accuracy target. All graphs are built with $M = 32$ and *efConstruction* between 200 (Sift1M) and 2,000 (CoCondenser); we sweep *efSearch* in $[10, 19]$ and λ in $[0.03, 0.15]$, and set the Zeta parameter κ to the value that maximizes compression without degrading query efficiency. Our implementation is written in Rust and integrated into kANNolo [26], which also provides the RerankIndex used for SnowFlake [32]. Experiments run on an Intel Xeon Silver 4314 CPU @ 2.40GHz.

Results Table 1 reports the comparison across the four datasets, with *Plain* as the uncompressed baseline. Bitpacker is a lightweight compromise, with 1.3×–1.6× savings at a latency close to the baseline. Elias–Fano is dominated on both axes: it is the slowest method everywhere and compresses no better than Bitpacker. Zeta achieves the smallest footprint on all datasets, from 1.7× (CoCondenser) to 2.3× (Sift1M), but this compactness costs up to ≈ 27% in latency on Sift1M and ≈ 13% on eBayProducts7M. Stream VByte is the only method that combines the two: its footprint stays within 10% of Zeta, and at 98% Accuracy@10 it is 1.7% faster than Plain on CoCondenser, 10.8% faster on eBayProducts7M, 0.6% faster on SnowFlake, and 2.7% slower on Sift1M, with the same ordering at the 90% and 94% cut-offs. Sift1M is the dataset where Stream VByte pays a marginal overhead, and it is also the smallest index, where the locality gains of reordering are weaker.

The EGB permutation adds a one-time offline cost, which accounts for 3.5% of the total build time on CoCondenser, 7.8% on SnowFlake, 15% on eBayProducts7M, and 30% on Sift1M. The fraction peaks on Sift1M because permutation time depends on the number of edges, while graph construction time grows with *efConstruction* and vector dimensionality, and Sift1M is low-dimensional and needs no high *efConstruction*. Since index construction is a one-time offline procedure, this overhead is a favorable trade for the memory reductions and the latency improvements it enables.

6. Conclusions and Ongoing Work

We studied the compression of the adjacency lists of proximity graphs for ANN search. An offline node permutation induces structural locality and substantially lowers the entropy of the lists under gap encoding, and Stream VByte compresses the reordered gaps into adjacency-list-aligned blocks that SIMD decodes at query time. Across four benchmarks the compression ratio reaches 2.1× with zero to negligible latency penalties, and the framework is in production at eBay. We are now extending it beyond HNSW, to graph indexes such as Vamana [33] and NSG [34], where the pruning rules and the degree distribution differ and the compression-latency trade-off may shift. A second direction is a compression-aware index that optimizes edge encoding and vector quantization jointly, which matters most where the two components contribute comparable shares of the index.

Declaration on Generative AI

During the preparation of this work, the authors used Gemini for grammar and spelling checking, paraphrasing, and rewording of existing text. Claude Opus 5 was used to assist in writing and debugging code. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

References

[1] F. Benocci, L. Dahlmann, F. Gargiulo, T. Heuer, S. Martinico, F. M. Nardini, C. Rulli, R. Venturini, Lossless compression of proximity graph indexes for efficient approximate nearest neighbors search, in: Proceedings of the 35th International ACM Conference on Knowledge and Information Management (CIKM), 2026.

[2] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, IEEE transactions on pattern analysis and machine intelligence 42 (2018) 824–836.

[3] M. Aumüller, E. Bernhardsson, A. Faithfull, Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms, Information Systems 87 (2020) 101374.

[4] M. Wang, X. Xu, Q. Yue, Y. Wang, A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search, arXiv preprint arXiv:2101.12631 (2021).

[5] H. Jégou, M. Douze, C. Schmid, Product quantization for nearest neighbor search, IEEE transactions on pattern analysis and machine intelligence 33 (2011) 117–28. doi:10.1109/TPAMI.2010.57.

[6] C. Aguerrebere, I. S. Bhati, M. Hildebrand, M. Tepper, T. Willke, Similarity search in the blink of an eye with compressed indices, Proc. VLDB Endow. 16 (2023) 3433–3446. URL: https://doi.org/10.14778/3611479.3611537. doi:10.14778/3611479.3611537.

[7] J. Gao, C. Long, Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search, Proc. ACM Manag. Data 2 (2024). URL: https://doi.org/10.1145/3654970. doi:10.1145/3654970.

[8] D. Severo, G. Ottaviano, M. Muckley, K. Ullrich, M. Douze, Lossless compression of vector ids for approximate nearest neighbor search, 2025. doi:10.48550/arXiv.2501.10479.

[9] J. Mackenzie, M. Petri, A. Moffat, Faster index reordering with bipartite graph partitioning, in: Proc. SIGIR, 2021, pp. 1910–1914.

[10] L. Gottesbüren, T. Heuer, P. Sanders, C. Schulz, D. Seemaier, Deep multilevel graph partitioning, in: 29th Annual European Symposium on Algorithms, ESA 2021, volume 204 of *LIPIcs*, Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, pp. 48:1–48:17. URL: https://doi.org/10.4230/LIPIcs.ESA.2021.48. doi:10.4230/LIPIcs.ESA.2021.48.

[11] D. Lemire, N. Kurz, C. Rupp, Stream vbyte: Faster byte-oriented integer compression, Information Processing Letters 130 (2018) 1–6.

[12] A. Trotman, J. Degenhardt, S. Kallumadi, The architecture of ebay search., in: eCOM@ SIGIR, 2017, p. 88.

[13] T. Cover, P. Hart, Nearest neighbor pattern classification, IEEE transactions on information theory 13 (1967) 21–27.

[14] H. Ferhatosmanoglu, E. Tuncel, D. Agrawal, A. El Abbadi, Approximate nearest neighbor searching in multimedia databases, in: Proceedings 17th International Conference on Data Engineering, IEEE, 2001, pp. 503–511.

[15] Y. Al-Jazzazi, H. Diwan, J. Gou, C. Musco, C. Musco, T. Suel, Distance adaptive beam search for provably accurate graph-based nearest neighbor search, ArXiv abs/2505.15636 (2025). URL: https://api.semanticscholar.org/CorpusID:278782177.

[16] P. Elias, Efficient storage and retrieval by content and address of static files, Journal of the ACM (JACM) 21 (1974) 246–260.

[17] J. Zobel, A. Moffat, Inverted files for text search engines, ACM Comput. Surv. 38 (2006) 6–es. URL: https://doi.org/10.1145/1132956.1132959. doi:10.1145/1132956.1132959.
[18] G. E. Pibiri, R. Venturini, Techniques for inverted index compression, ACM Computing Surveys (CSUR) 53 (2021) 125:1–125:36. URL: https://doi.org/10.1145/3415148. doi:10.1145/3415148.
[19] D. A. Huffman, A method for the construction of minimum-redundancy codes, Proceedings of the IRE 40 (2007) 1098–1101.
[20] J. Duda, Asymmetric numeral systems, arXiv preprint arXiv:0902.0271 (2009).
[21] P. Elias, Universal codeword sets and representations of the integers, IEEE transactions on information theory 21 (2003) 194–203.
[22] P. Boldi, S. Vigna, Codes for the world wide web, Internet mathematics 2 (2005) 407–429.
[23] P. Boldi, S. Vigna, The webgraph framework i: compression techniques, in: Proceedings of the 13th international conference on World Wide Web, ACM, 2004, pp. 595–602.
[24] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with gpus, IEEE transactions on big data 7 (2019) 535–547.
[25] B. Coleman, S. Segarra, A. Smola, A. Shrivastava, Graph reordering for cache-efficient near neighbor search, in: Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22, Curran Associates Inc., Red Hook, NY, USA, 2022.
[26] L. Delfino, D. Erriquez, S. Martinico, F. M. Nardini, C. Rulli, R. Venturini, kannolo: Sweet and smooth approximate k-nearest neighbors search, in: Advances in Information Retrieval: 47th European Conference on Information Retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, Proceedings, Part IV, Springer-Verlag, Berlin, Heidelberg, 2025, p. 400–406. URL: https://doi.org/10.1007/978-3-031-88717-8_29. doi:10.1007/978-3-031-88717-8_29.
[27] D. Lowe, Object recognition from local scale-invariant features, in: Proceedings of the Seventh IEEE International Conference on Computer Vision, volume 2, 1999, pp. 1150–1157 vol.2. doi:10.1109/ICCV.1999.790410.
[28] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, L. Deng, MS MARCO: A human generated machine reading comprehension dataset, in: Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches, volume 1773, 2016. URL: https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf.
[29] P. Yu, L. Merrick, G. Nuti, D. Campos, Arctic-embed 2.0: Multilingual retrieval without compromise, arXiv preprint arXiv:2412.04506 (2024).
[30] T. Formal, C. Lassance, B. Piwowarski, S. Clinchant, From distillation to hard negative sampling: Making sparse neural ir models more effective, in: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2022, p. 2353–2359. doi:10.1145/3477495.3531857.
[31] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2019, pp. 4171–4186. doi:10.18653/v1/N19-1423.
[32] S. Martinico, F. M. Nardini, C. Rulli, R. Venturini, Multivector reranking in the era of strong first-stage retrievers, arXiv preprint arXiv:2601.05200 (2026). URL: https://arxiv.org/abs/2601.05200. arXiv:2601.05200.
[33] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, R. Kadekodi, Diskann: Fast accurate billion-point nearest neighbor search on a single node, Advances in neural information processing Systems 32 (2019).
[34] C. Fu, C. Xiang, C. Wang, D. Cai, Fast approximate nearest neighbor search with the navigating spreading-out graph, arXiv preprint arXiv:1707.00143 (2017).